

# Natural Language Processing (NLP) with Python

## Table of Contents:

1. [What is Natural Language Processing?](#)
2. [Applications of NLP](#)
3. [Understanding Natural Language Processing \(NLP\)](#)
4. [Rule-based NLP vs. Statistical NLP](#)
5. [Components of Natural Language Processing \(NLP\)](#)
6. [Current challenges in NLP](#)
7. [Easy to Use NLP Libraries](#)
8. [Exploring Features of NLTK](#)
9. [Word Cloud](#)

## Exploring Features of NLTK:

### a. Open the text file for processing:

First, we are going to open and read the file which we want to analyze.

```
#open the text file
text_file = open("T1.txt")
text = text_file.read()
print(type(text))
print("\n")
print(text)
print(len(text))
```

### b. Import required libraries:

For various data processing cases in NLP, we need to import some libraries. In this case, we are going to use NLTK for Natural Language Processing. We will use it to perform various operations on the text.

```
#import required libraries
import nltk
from nltk import sent_tokenize
from nltk import word_tokenize
```

### c. Sentence tokenizing:

By tokenizing the text with `sent_tokenize()`, we can get the text as sentences.

```
# tokenize the text by sentences
sentences = sent_tokenize(text)
#how many sentences are there?
print(len(sentences))
#print the sentences
print(sentences)
```

In the example above, we can see the entire text of our data is represented as sentences and also notice that the total number of sentences here is **9**.

### d. Word tokenizing:

By tokenizing the text with `word_tokenize()`, we can get the text as words.

```
words = word_tokenize(text)
print(len(words))
print(words)
```

Next, we can see the entire text of our data is represented as words and also notice that the total number of words here is **144**.

### e. Find the frequency distribution:

Let's find out the frequency of words in our text.

```
#import required libraries
from nltk.probability import FreqDist
#Find the frequency
fdist = FreqDist(words)
#print 10 most common words
fdist.most_common(10)
```

Notice that the most used words are punctuation marks and stopwords. We will have to remove such words to analyze the actual text.

### f. Plot the frequency graph:

Let's plot a graph to visualize the word distribution in our text.

```
import matplotlib.pyplot as plt
```

```
fdist.plot(10)
```

In the graph above, notice that a period “.” is used nine times in our text. Analytically speaking, punctuation marks are not that important for natural language processing. Therefore, in the next step, we will be removing such punctuation marks.

#### **g. Remove punctuation marks:**

Next, we are going to remove the punctuation marks as they are not very useful for us. We are going to use `isalpha()` method to separate the punctuation marks from the actual text. Also, we are going to make a new list called `words_no_punc`, which will store the words in lower case but exclude the punctuation marks.

```
words_no_punc = []
#removing punctuation marks
for w in words:
    if w.isalpha():
        words_no_punc.append(w.lower())
#print the words without punctuation marks
print(words_no_punc)
print("\n")
#length
print(len(words_no_punc))
```

As shown above, all the punctuation marks from our text are excluded. These can also cross-check with the number of words.

#### **h. Plotting graph without punctuation marks:**

```
fdist = FreqDist(words_no_punc)
fdist.most_common(10)
#plot the most common words on graph
fdist.plot(10)
```

Notice that we still have many words that are not very useful in the analysis of our text file sample, such as “and,” “but,” “so,” and others. Next, we need to remove coordinating conjunctions.

#### **i. List of stopwords:**

```
from nltk.corpus import stopwords
#list of stopwords
stopwords = stopwords.words("english")
print(stopwords)
```

#### j. Removing stopwords:

```
# Empty list to store clean words
clean_words = []

for w in words_no_punc:
    if w not in stopwords:
        clean_words.append(w)

print(clean_words)
print("\n")
print(len(clean_words))
```

#### k. Final frequency distribution:

```
#Frequency distribution
fdist = FreqDist(clean_words)
fdist.most_common(10)

fdist.plot(10) #plot the most common words on graph
```

As shown above, the final graph has many useful words that help us understand what our sample data is about, showing how essential it is to perform data cleaning on NLP.

Next, we will cover various topics in NLP with coding examples.

#### Word Cloud:

Word Cloud is a data visualization technique. In which words from a given text display on the main chart. In this technique, more frequent or essential words display in a larger and bolder font, while less frequent or essential words display in smaller or thinner fonts. It is a beneficial technique in NLP that gives us a glance at what text should be analyzed.

#### Properties:

1. **font\_path**: It specifies the path for the fonts we want to use.
2. **width**: It specifies the width of the canvas.
3. **height**: It specifies the height of the canvas.
4. **min\_font\_size**: It specifies the smallest font size to use.

5. **max\_font\_size:** It specifies the largest font size to use.
6. **font\_step:** It specifies the step size for the font.
7. **max\_words:** It specifies the maximum number of words on the word cloud.
8. **stopwords:** Our program will eliminate these words.
9. **background\_color:** It specifies the background color for canvas.
10. **normalize\_plurals:** It removes the trailing “s” from words.

Read the full documentation on [WordCloud](#).

#### Word Cloud Python Implementation:

```
#Library to form wordcloud
from wordcloud import WordCloud

#library to plot the wordcloud
import matplotlib.pyplot as plt

#generating the wordcloud
wordcloud = WordCloud().generate(text)

#plot the wordcloud
plt.figure(figsize =(12,12))
plt.imshow(wordcloud)

#to remove thw axis value
plt.axis("off")
plt.show()
```

As shown in the graph above, the most frequent words display in larger fonts. The word cloud can be displayed in any shape or image.

